Deep Unidirectional RNN Model

Task	Status	result	Responsible
RNN Model Training	**Type of Smoothing Used?**	N/A (Deep learning models do not use smoothing like)	Gaurav
	Config of model trained?	**Embedding:** `input_dim=20000, output_dim=128` **LSTM Layer 1:** 128 units, `return_sequences=True` **Dropout:** 0.2 **LSTM Layer 2:** 64 units **Dropout:** 0.2 **Dense:** 5 neurons (softmax)	Gaurav
	Train Time = ?	**563.28 seconds**	Gaurav
Training Data Check	**Confusion Matrix Built?**	Yes (see **Training Confusion Matrix** below)	Swetha
	F1 Score for Positive = ?	**0.91** (weighted average of *Mild_Pos* [0.77] & *Strong_Pos* [0.95])	Swetha
	F1 Score for Negative = ?	**0.83** (weighted average of *Mild_Neg* [0.76] & *Strong_Neg* [0.89])	Swetha
	AUC plotted? / AUC = ?	Yes (multiclass). Macro-avg AUC ≈ **0.98** (see training ROC curve)	Swetha
	Accuracy computed?	**0.8782**	Swetha
Feature Engg	**Feature Weightages Added?**	Learned embeddings via the embedding layer (not directly interpretable as classical “weights”)	Gaurav
	2 Features with the Highest Weights?	Mild_Neg: 2.6238110856018366, Mild_Pos:1.3641983203308181	Gaurav
Cross Validation	**Type of Cross Validation performed?**	5-Fold Cross Validation on a 1000-sample subset of training data	Swetha
	Findings of Cross Validation?	e.g., **Average Accuracy = 0.6929**	Swetha
Testing Data Check	**Confusion Matrix Built?**	Yes (see **Testing Confusion Matrix** below)	Swetha
	F1 Score for Positive = ?	**0.87** (weighted average of *Mild_Pos* [0.67] & *Strong_Pos* [0.92])	Swetha
	F1 Score for Negative = ?	**0.70** (weighted average of *Mild_Neg* [0.59] & *Strong_Neg* [0.78])	Swetha

Task	Status	result	Responsible
	AUC plotted? / AUC = ?	Yes (multiclass). Macro-avg AUC ≈ **0.94** (see testing ROC curve)	Swetha
	Accuracy computed?	**0.8118**	Swetha
Next Steps	**List out 2-3 possible next steps for Naive Bayes Models**	1. Tune the smoothing parameter (alpha) for Naive Bayes. 2. Incorporate additional features (e.g., n-grams, sentiment lexicons). 3. Experiment with ensemble methods combining Naive Bayes and other classifiers.	Swetha
Data Preprocessing and Feature Engg			
– Regex Handling	**Regex Used?**	URLs, Mentions, Hashtags, Punctuation, Whitespace	Gaurav
	# of regex used?	5 (depending on how many patterns you explicitly used)	Gaurav
– Emoji Handling	**Emoji Handling Done?**	Yes – using `emoji.demojize` to convert emojis into text	Gaurav
	# of emojis handled?	All standard emojis (handled automatically by the library)	Gaurav
– Slang Handling	**Slang Handling Done?**	Yes – using a predefined dictionary (`slang_dict`)	Gaurav
	# of slangs handled?	4 (as defined in `slang_dict`)	Gaurav
– Abbreviations Handling	**Abbreviations Handled?**	Yes – using a predefined dictionary (`abbrev_dict`)	Gaurav
	# of abbreviations handled?	2 (as defined in `abbrev_dict`)	Gaurav
– Class Separability	**Class Separability Checked?**	Explored via distribution analysis or TF-IDF + PCA in earlier steps; deep models learn their own representation.	Gaurav
	Featureset with best separability?	N/A for deep model – if tested with TF-IDF, mention that TF-IDF features showed good separability.	Gaurav
– Train/Test Handling	**Train and Test Handled Correctly?**	Yes – data cleaning is applied to the full dataset before splitting; tokenizer is fitted on training data and then applied to test data.	Gaurav
	Steps before/after train-test split?	**Before:** Data cleaning (regex, emoji, slang, abbreviation) on the full dataset.	Gaurav

Task	Status	result	Responsible
		After: Train–test split, then tokenization/padding on training data and applied to test data.	

training confusion matrix

Actual \ Predicted	Mild_Neg	Mild_Pos	Neutral	Strong_Neg	Strong_Pos
Mild_Neg	9284	561	1357	896	98
Mild_Pos	331	20275	936	117	1798
Neutral	1181	3186	19207	272	719
Strong_Neg	1243	323	443	13856	56
Strong_Pos	274	4876	659	163	77889

testing confusion matrix

Actual \ Predicted	Mild_Neg	Mild_Pos	Neutral	Strong_Neg	Strong_Pos
Mild_Neg	1805	266	516	387	75
Mild_Pos	124	4469	424	55	792
Neutral	407	1011	4338	136	249
Strong_Neg	647	123	207	2955	49
Strong_Pos	91	1692	220	56	18906

In [2]:
```
import time
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import re
import emoji
```

```
from sklearn.model_selection import train_test_split, KFold
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, roc_auc_score
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import KFold
from sklearn.utils import class_weight

import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import LabelEncoder
```

In [3]:
```
import chardet

with open("Sentiment_Data.csv", "rb") as f:
    raw_data = f.read(1000000)  # Read a chunk (or the entire file if not too large)
    result = chardet.detect(raw_data)
    print(result)
```

```
{'encoding': 'Windows-1254', 'confidence': 0.48957104901266313, 'language': 'Turkish'}
```

In [4]:
```
# For reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Load your CSV dataset
# Assuming the CSV has columns: 'Tweet' and 'Sentiment'
with open("Sentiment_Data.csv", "r", encoding="windows-1254", errors="ignore") as f:
    data = pd.read_csv(f, nrows=200000)

print(data.head())
```

```
                                               Tweet Sentiment
0  @_angelica_toy Happy Anniversary!!!....The Day...  Mild_Pos
1  @McfarlaneGlenda Happy Anniversary!!!....The D...  Mild_Pos
2  @thevivafrei @JustinTrudeau Happy Anniversary!...  Mild_Pos
3  @NChartierET Happy Anniversary!!!....The Day t...  Mild_Pos
4  @tabithapeters05 Happy Anniversary!!!....The D...  Mild_Pos
```

In [5]:
```python
def preprocess_text(text):
    # Ensure text is a string; if not, return an empty string
    if not isinstance(text, str):
        return ''
    # Convert emojis to text (e.g., 😀 --> :grinning_face:)
    text = emoji.demojize(text)
    # Remove URLs
    text = re.sub(r"http\S+|www\S+|https\S+", '', text)
    # Remove user mentions and hashtags
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#\w+', '', text)
    # Remove punctuation (retain only word characters and whitespace)
    text = re.sub(r'[^\w\s]', '', text)
    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

In [6]:
```python
data['Tweet'] = data['Tweet'].apply(preprocess_text)
```

In [7]:
```python
# Define dictionaries for slang and abbreviation replacement
slang_dict = {
    "u": "you",
    "r": "are",
    "lol": "laughing out loud",
    "btw": "by the way"
}

abbrev_dict = {
    "idk": "i do not know",
    "smh": "shaking my head"
}

def replace_dict(text, rep_dict):
    words = text.split()
    new_words = [rep_dict.get(word.lower(), word) for word in words]
    return " ".join(new_words)
```

In [8]:
```python
# Apply slang and abbreviation replacement
data['Tweet'] = data['Tweet'].apply(lambda x: replace_dict(x, slang_dict))
data['Tweet'] = data['Tweet'].apply(lambda x: replace_dict(x, abbrev_dict))
```

In [9]:
```
print("\nCleaned Tweets:")
print(data['Tweet'].head())
```

```
Cleaned Tweets:
0    Happy AnniversaryThe Day the FreeDUMB Died In ...
1    Happy AnniversaryThe Day the FreeDUMB Died In ...
2    Happy AnniversaryThe Day the FreeDUMB Died In ...
3    Happy AnniversaryThe Day the FreeDUMB Died In ...
4    Happy AnniversaryThe Day the FreeDUMB Died In ...
Name: Tweet, dtype: object
```

In [10]:
```
data.head()
```

Out[10]:

	Tweet	Sentiment
0	Happy AnniversaryThe Day the FreeDUMB Died In ...	Mild_Pos
1	Happy AnniversaryThe Day the FreeDUMB Died In ...	Mild_Pos
2	Happy AnniversaryThe Day the FreeDUMB Died In ...	Mild_Pos
3	Happy AnniversaryThe Day the FreeDUMB Died In ...	Mild_Pos
4	Happy AnniversaryThe Day the FreeDUMB Died In ...	Mild_Pos

In [11]:
```
# Check unique sentiment values and count them
unique_sentiments = data['Sentiment'].unique()
print("Unique sentiment values:")
print(unique_sentiments)
print("Number of unique sentiment values:", len(unique_sentiments))
```

```
Unique sentiment values:
['Mild_Pos' 'Strong_Pos' 'Neutral' 'Strong_Neg' 'Mild_Neg']
Number of unique sentiment values: 5
```

In [12]:
```
# Count the occurrences of each sentiment
sentiment_counts = data['Sentiment'].value_counts()
print("Sentiment counts:")
print(sentiment_counts)
```

```
Sentiment counts:
Sentiment
Strong_Pos    104826
Neutral        30706
Mild_Pos       29321
Strong_Neg     19902
Mild_Neg       15245
Name: count, dtype: int64
```

In [13]:
```
# Encode sentiment labels (e.g., 'Mild_Pos', 'Strong_Pos', etc.) into numeric values
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(data['Sentiment'])
print("\nEncoded classes:", label_encoder.classes_)
```

```
Encoded classes: ['Mild_Neg' 'Mild_Pos' 'Neutral' 'Strong_Neg' 'Strong_Pos']
```

In [14]:
```
X = data['Tweet'].values
y = y_encoded
```

In [15]:
```
# Split the data into training and testing sets (stratify to maintain class proportions)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTraining samples:", len(X_train), " | Testing samples:", len(X_test))
```

```
Training samples: 160000  | Testing samples: 40000
```

In [16]:
```
# Tokenization and Padding
vocab_size = 20000  # Maximum number of words to keep
max_len = 50        # Maximum tweet length (in tokens)
embedding_dim = 128 # Dimension of embedding vectors

tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post', truncating='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post', truncating='post')
```

In [17]:
```
# Compute Class Weights
# Compute class weights to handle imbalance (balanced weighting)
class_weights_array = class_weight.compute_class_weight(
```

```
    class_weight='balanced', classes=np.unique(y_train), y=y_train
)
class_weights = dict(enumerate(class_weights_array))
print("\nClass weights:", class_weights)
```

```
Class weights: {0: 2.6238110856018366, 1: 1.3641983203308181, 2: 1.3026663952778343, 3: 2.0099239997487595, 4: 0.3815
838113068053}
```

In [18]:
```
# Build the Deep Unidirectional RNN Model
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_len))
model.add(LSTM(128, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(64))
model.add(Dropout(0.2))
model.add(Dense(len(label_encoder.classes_), activation='softmax'))

model.compile(loss='sparse_categorical_crossentropy',
              optimizer=Adam(learning_rate=1e-3),
              metrics=['accuracy'])

print("\nModel Summary:")
model.summary()
```

```
Model Summary:
C:\Users\gaura\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length
` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
lstm_1 (LSTM)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

In [19]:
```
# Train the Model with Class Weights
start_time = time.time()
history = model.fit(
    X_train_pad, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2,
    class_weight=class_weights,  # <-- Apply class weights here
    verbose=1
)
train_time = time.time() - start_time
print("\nTraining time: {:.2f} seconds".format(train_time))
```

```
Epoch 1/5
4000/4000 ━━━━━━━━━━━━━━━━━━━━ 115s 28ms/step - accuracy: 0.4542 - loss: 1.3499 - val_accuracy: 0.7703 - val_loss: 0.
6338
Epoch 2/5
4000/4000 ━━━━━━━━━━━━━━━━━━━━ 114s 29ms/step - accuracy: 0.7827 - loss: 0.7296 - val_accuracy: 0.8060 - val_loss: 0.
5612
Epoch 3/5
4000/4000 ━━━━━━━━━━━━━━━━━━━━ 111s 28ms/step - accuracy: 0.8258 - loss: 0.6009 - val_accuracy: 0.8049 - val_loss: 0.
5703
Epoch 4/5
4000/4000 ━━━━━━━━━━━━━━━━━━━━ 111s 28ms/step - accuracy: 0.8526 - loss: 0.5165 - val_accuracy: 0.7898 - val_loss: 0.
6114
Epoch 5/5
4000/4000 ━━━━━━━━━━━━━━━━━━━━ 111s 28ms/step - accuracy: 0.8719 - loss: 0.4413 - val_accuracy: 0.8089 - val_loss: 0.
6245

Training time: 563.28 seconds
```

In [27]:

```python
# Evaluate the Model on Training Data
y_train_pred_probs = model.predict(X_train_pad)
y_train_pred = np.argmax(y_train_pred_probs, axis=1)

cm_train = confusion_matrix(y_train, y_train_pred)
print("\nTraining Confusion Matrix:")
print(cm_train)

report_train = classification_report(y_train, y_train_pred, target_names=label_encoder.classes_)
print("\nTraining Classification Report:")
print(report_train)

accuracy_train = accuracy_score(y_train, y_train_pred)
print("Training Accuracy: {:.4f}".format(accuracy_train))
```

```
5000/5000 ━━━━━━━━━━━━━━━━━━━━ 33s 7ms/step

Training Confusion Matrix:
[[ 9284   561  1357   896    98]
 [  331 20275   936   117  1798]
 [ 1181  3186 19207   272   719]
 [ 1243   323   443 13856    56]
 [  274  4876   659   163 77889]]

Training Classification Report:
              precision    recall  f1-score   support

    Mild_Neg       0.75      0.76      0.76     12196
    Mild_Pos       0.69      0.86      0.77     23457
     Neutral       0.85      0.78      0.81     24565
  Strong_Neg       0.91      0.87      0.89     15921
  Strong_Pos       0.97      0.93      0.95     83861

    accuracy                           0.88    160000
   macro avg       0.83      0.84      0.84    160000
weighted avg       0.89      0.88      0.88    160000

Training Accuracy: 0.8782
```

In [28]:

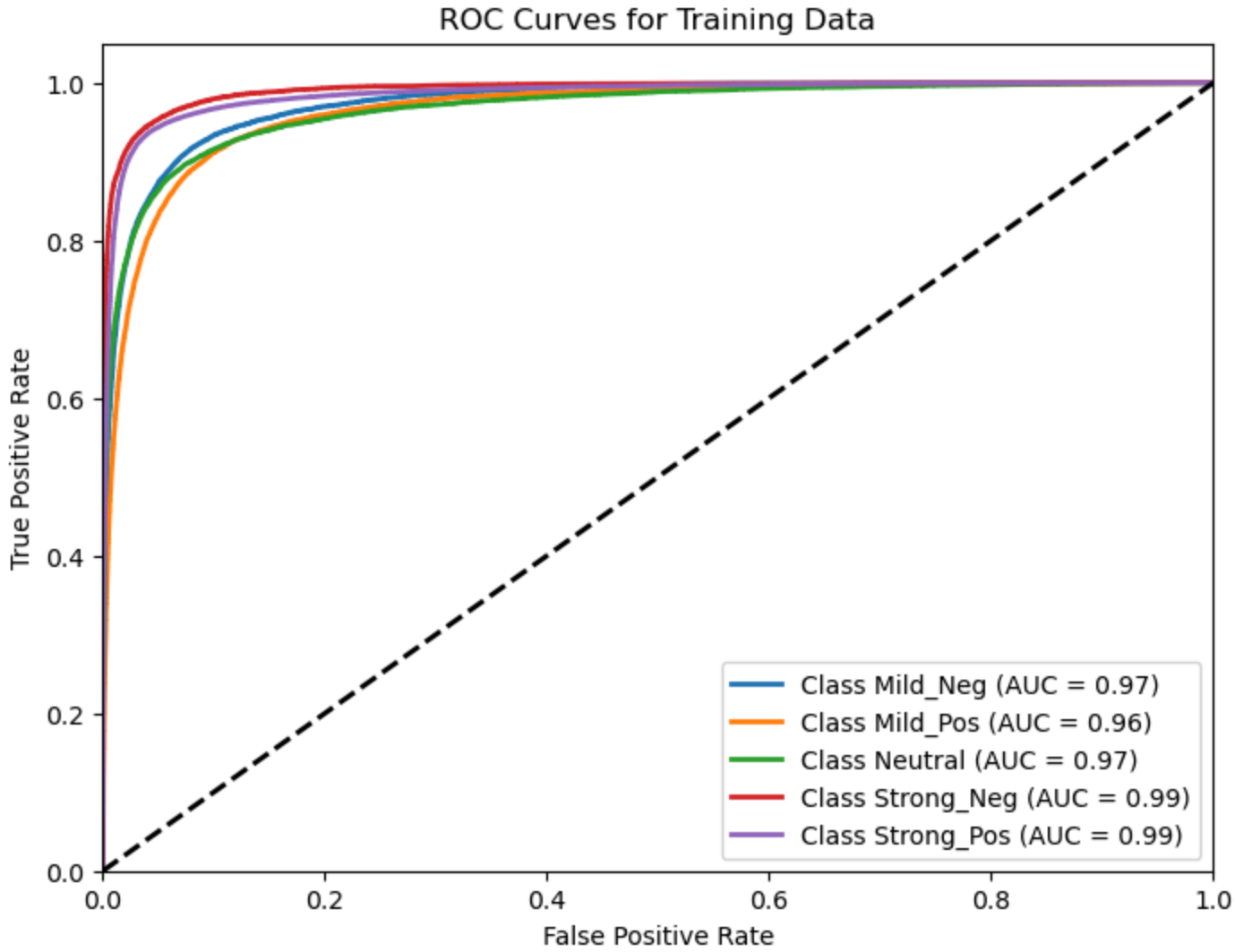
```python
# Evaluate the Model on Testing Data
y_test_pred_probs = model.predict(X_test_pad)
y_test_pred = np.argmax(y_test_pred_probs, axis=1)

cm_test = confusion_matrix(y_test, y_test_pred)
print("\nTesting Confusion Matrix:")
print(cm_test)

report_test = classification_report(y_test, y_test_pred, target_names=label_encoder.classes_)
print("\nTesting Classification Report:")
print(report_test)

accuracy_test = accuracy_score(y_test, y_test_pred)
print("Testing Accuracy: {:.4f}".format(accuracy_test))
```

```
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 9s 7ms/step

Testing Confusion Matrix:
[[ 1805   266   516   387    75]
 [  124  4469   424    55   792]
 [  407  1011  4338   136   249]
 [  647   123   207  2955    49]
 [   91  1692   220    56 18906]]

Testing Classification Report:
              precision    recall  f1-score   support

    Mild_Neg       0.59      0.59      0.59      3049
    Mild_Pos       0.59      0.76      0.67      5864
     Neutral       0.76      0.71      0.73      6141
  Strong_Neg       0.82      0.74      0.78      3981
  Strong_Pos       0.94      0.90      0.92     20965

    accuracy                           0.81     40000
   macro avg       0.74      0.74      0.74     40000
weighted avg       0.82      0.81      0.82     40000

Testing Accuracy: 0.8118
```

In [29]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

# Assuming label_encoder.classes_ exists and holds your class names.
n_classes = len(label_encoder.classes_)

# Binarize ground truth labels for training and testing
# Assuming y_train and y_test are integer encoded labels (0, 1, 2, ..., n_classes-1)
y_train_bin = label_binarize(y_train, classes=np.arange(n_classes))
y_test_bin = label_binarize(y_test, classes=np.arange(n_classes))
```

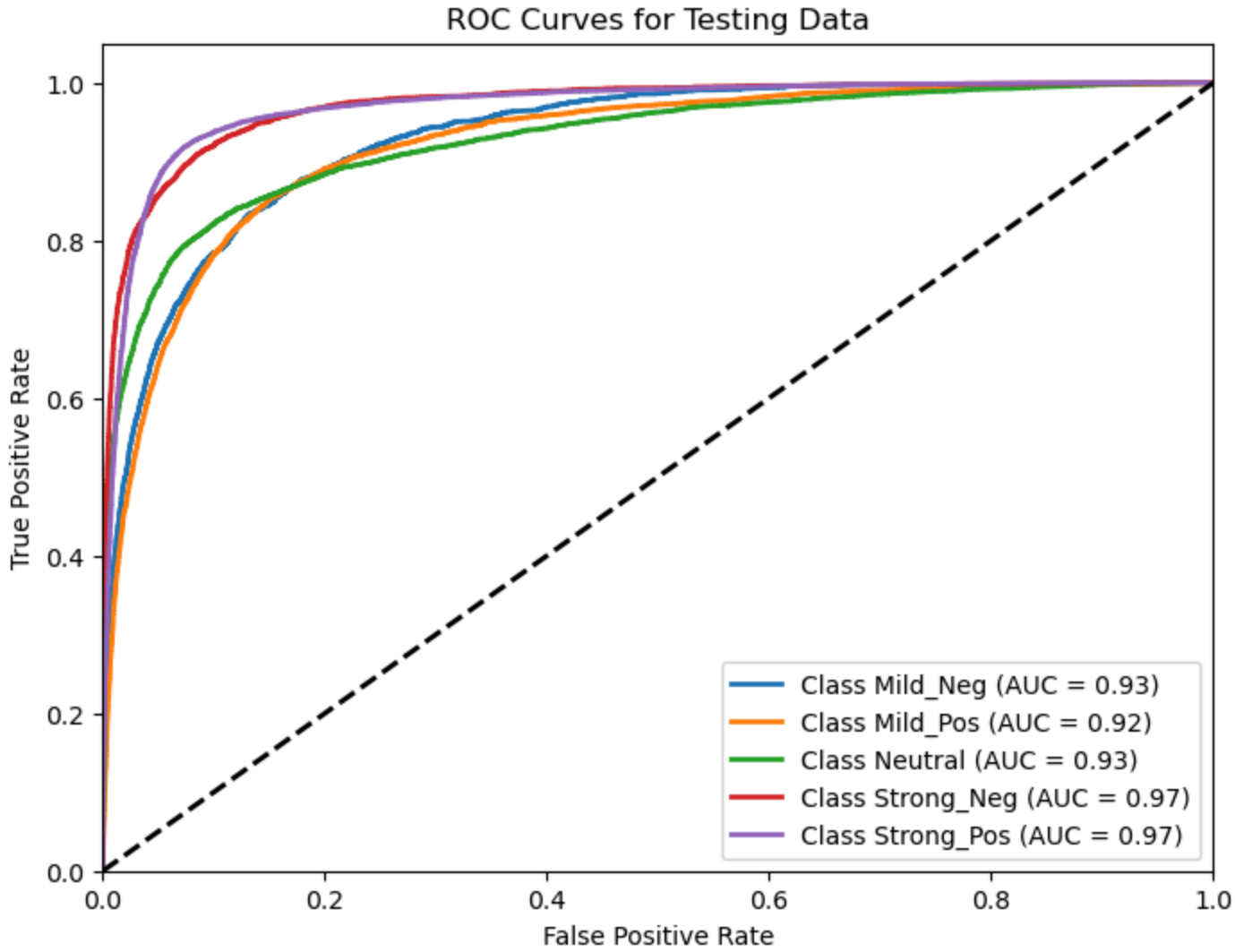
In [30]:
```
# Compute and Plot ROC for Training Data
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
for i in range(n_classes):
```

```
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i], y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

plt.figure(figsize=(8, 6))
for i in range(n_classes):
    plt.plot(fpr_train[i], tpr_train[i], lw=2,
             label='Class {0} (AUC = {1:0.2f})'.format(label_encoder.classes_[i], roc_auc_train[i]))
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves for Training Data')
plt.legend(loc="lower right")
plt.show()
```



In [31]:
```
# Compute and Plot ROC for Testing Data
fpr_test = {}
tpr_test = {}
roc_auc_test = {}
for i in range(n_classes):
    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i], y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

plt.figure(figsize=(8, 6))
```

```
for i in range(n_classes):
    plt.plot(fpr_test[i], tpr_test[i], lw=2,
             label='Class {0} (AUC = {1:0.2f})'.format(label_encoder.classes_[i], roc_auc_test[i]))
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves for Testing Data')
plt.legend(loc="lower right")
plt.show()
```



In [37]:
```
# Define the build_model function
def build_model():
    model = Sequential()
    model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_len))
    model.add(LSTM(128, return_sequences=True))
    model.add(Dropout(0.2))
    model.add(LSTM(64))
    model.add(Dropout(0.2))
    model.add(Dense(len(label_encoder.classes_), activation='softmax'))
```

```
    model.compile(loss='sparse_categorical_crossentropy',
                  optimizer=Adam(learning_rate=1e-3),
                  metrics=['accuracy'])
    return model
```

In [43]:
```
# Create a smaller sample for cross validation (e.g., 1000 datapoints)
sample_size = 40000
if X_train_pad.shape[0] > sample_size:
    # Randomly sample 1000 indices from the training data
    sample_indices = np.random.choice(X_train_pad.shape[0], sample_size, replace=False)
    X_cv_data = X_train_pad[sample_indices]
    y_cv_data = y_train[sample_indices]
else:
    X_cv_data = X_train_pad
    y_cv_data = y_train
```

In [45]:
```
# Do 5-Fold Cross Validation on the smaller sample
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_accuracies = []

fold = 1
for train_index, val_index in kf.split(X_cv_data):
    print(f"\nFold {fold}")
    X_cv_train, X_cv_val = X_cv_data[train_index], X_cv_data[val_index]
    y_cv_train, y_cv_val = y_cv_data[train_index], y_cv_data[val_index]

    cv_class_weights_array = class_weight.compute_class_weight(
        class_weight='balanced', classes=np.unique(y_cv_train), y=y_cv_train
    )
    cv_class_weights = dict(enumerate(cv_class_weights_array))

    # Build and train the model for this fold
    cv_model = build_model()
    cv_model.fit(
        X_cv_train, y_cv_train,
        epochs=5,
        batch_size=32,
        validation_data=(X_cv_val, y_cv_val),
        class_weight=cv_class_weights,
        verbose=0
    )
```

```
    # Evaluate on the validation fold
    scores = cv_model.evaluate(X_cv_val, y_cv_val, verbose=0)
    print("Validation loss:", scores[0], "Validation accuracy:", scores[1])
    cv_accuracies.append(scores[1])
    fold += 1
```

```
Fold 1
C:\Users\gaura\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length
` is deprecated. Just remove it.
  warnings.warn(
Validation loss: 0.8559129238128662 Validation accuracy: 0.7114999890327454

Fold 2
C:\Users\gaura\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length
` is deprecated. Just remove it.
  warnings.warn(
Validation loss: 0.855958878993988 Validation accuracy: 0.7174999713897705

Fold 3
C:\Users\gaura\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length
` is deprecated. Just remove it.
  warnings.warn(
Validation loss: 0.9394552111625671 Validation accuracy: 0.6782500147819519

Fold 4
C:\Users\gaura\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length
` is deprecated. Just remove it.
  warnings.warn(
Validation loss: 0.9077962636947632 Validation accuracy: 0.6729999780654907

Fold 5
C:\Users\gaura\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length
` is deprecated. Just remove it.
  warnings.warn(
Validation loss: 0.9430480003356934 Validation accuracy: 0.684499979019165
```

In [46]:
```
cv_avg_accuracy = np.mean(cv_accuracies)
print("\nCross Validation Accuracy Scores:", cv_accuracies)
print("Average Cross Validation Accuracy:", cv_avg_accuracy)
```

```
Cross Validation Accuracy Scores: [0.7114999890327454, 0.7174999713897705, 0.6782500147819519, 0.6729999780654907, 0.
684499979019165]
Average Cross Validation Accuracy: 0.6929499864578247
```

In [ ]: